

0-k BFS – Speaker Script

Companion text to `presentation.html`

Makar Artemov, Mark Shkut
Skoltech, Algorithms 2026

May 21, 2026

Literal text to read for each slide of `presentation.html`. One section here = one slide. Read the paragraph aloud, then advance to the next slide when you reach the dashed line.

1. Title – “0-k BFS”

“Good afternoon. I’m Makar, this is Mark. We’ve built a C++17 implementation of the 0-k BFS family of shortest-path algorithms and benchmarked it against Dijkstra as our Stage 1 deliverable. The talk is about fifteen minutes; questions at the end.”

2. Problem statement

“The problem is single-source shortest path on a graph whose edge weights are non-negative integers from $\{0, 1, \dots, k\}$, where k is a small known constant – typically four, eight, or sixteen. For arbitrary weights the classical answer is Dijkstra at $O((V + E) \log V)$. The point of the talk is: when the weight range is small and known, we can drop the log factor. Our Stage 1 goal was to build that implementation, compare it against a Dijkstra baseline, and push the input size as close to 10^9 vertices as a single commodity workstation can handle. The applications that motivated this are road networks with quantised travel times, grid pathfinding in games and robotics, and the BFS family that’s standard in competitive programming.”

3. Headline result – 200×200 grid

“Before any theory, here’s the punchline on a small example. Same 200×200 grid, $k = 8$, the same source. Both algorithms produce the same distance field – the colour pattern is identical. The numbers in the panel titles are real measured wall-clock times: 0-k BFS finishes in four milliseconds, Dijkstra in thirteen point three – so Dijkstra spends 3.32 times more wall-clock time getting to the same answer.”

4. The big number – $V = 9 \cdot 10^8$

“Now the headline at scale. We took an implicit grid of thirty thousand by thirty thousand – nine hundred million vertices, three point six billion edges – and ran a single-source shortest path on a single thread, on a workstation with thirty-two gigabytes of RAM. Dijkstra: six hundred nineteen seconds. The deque-based 0-1 BFS: one hundred twenty-one seconds. 5.12 times faster. Byte-identical distance-sum checksum, so correctness is verified. The two tricks that let this fit in commodity RAM are an implicit grid – we never store edges, we compute neighbours in $O(1)$ – and a 32-bit distance type, which is safe because $k \cdot V$ fits in 2^{32} .”

5. Three algorithms

“We implemented three algorithms behind one abstraction. The baseline, `dijkstra_pq`, is a standard `std::priority_queue` with lazy decrease-key. Then `bfs_01`, the 0-1 BFS via `std::deque`. And `bfs_0k`, which uses $k+1$ circular FIFO queues for the general weight range. All three are templated over a graph adapter, so exactly the same algorithm code runs on the implicit `GridGraph` – which scales to $V \approx 10^9$ – and on the classical `GraphCSR` – which scales to $V \approx 10^8$ because of the edge

storage.”

6. 0–1 BFS: a deque

“The 0–1 BFS is the simplest case in the family. With weights zero or one, a single double-ended queue is enough. Pop from the front; relax each neighbour; if the edge weight is zero, push the new vertex to the front – it belongs to the same frontier as the current one; if the weight is one, push to the back – next frontier. Invariant: at any moment the deque holds at most two adjacent distance levels. The $\log V$ from Dijkstra is gone: every operation is $O(1)$ amortised. The check `dist[u] < cur_dist_of(u)` skips stale entries – a vertex can be discovered several times before its distance is final.”

7. 0– k BFS: circular FIFOs

“Generalize from weights $\{0, 1\}$ to $\{0, 1, \dots, k\}$. The key observation is that a relaxation moves a vertex at most k steps forward in distance order. So only $k+1$ distance layers are ever alive at once. We allocate $k+1$ FIFO queues indexed by distance modulo $k+1$ – a circular buffer of buckets. The `cur` pointer walks forward through them; when the active bucket is empty, we advance `cur` and the next bucket becomes active. A relaxation pushes the target into bucket `(new_dist) mod M`, which is sometimes a different bucket than the old slot – so the old slot just becomes a stale entry that gets skipped on pop. `cur` only ever moves forward – that’s why the structure is circular and doesn’t need to be a heap.”

8. Queue mechanics, step by step

“This is the pedagogical visualization of the data structure. Tiny example: 10×10 grid, $k = 3$. On the right are the four real queues, $Q[0]$ through $Q[3]$, with their actual contents – the numbers inside the cells are vertex ids. On the left is the grid; each cell is coloured by which bucket currently holds it. The red `cur` arrow marks the active bucket. Watch: each relaxation puts the vertex into the bucket chosen by `(new distance) mod four`; the active bucket drains in FIFO order; when it’s empty, `cur` advances; stale entries left over from earlier relaxations just get skipped on pop. That’s all the algorithm is.”

9. 0– k BFS wavefront – 120×120 , $k=4$

“On a larger grid with $k = 4$ and source near the centre, the frontier looks like concentric distance rings. Same shape as a Dijkstra wavefront – but without the per-pop log overhead. The gradient pattern is shaped by random terrain weights, which is why the rings aren’t perfect circles.”

10. Datasets

“The benchmarks use four synthetic generators, all driven by a seeded `std::mt19937_64` so every run is reproducible. The implicit grid – our main backend – assigns each vertex a ‘terrain’ byte and derives edge weights from neighbouring terrain values, producing structured ‘mountains and valleys’. Then Erdős–Rényi $G(n, p)$ with weights uniform in $\{0..k\}$, a layered DAG with L layers of width W and given fanout, and an adversarial chain graph with random chords. On the grid backend edges are computed on the fly – that’s the trick that lets V reach 10^9 without buying more RAM.”

11. Distance-field visualisation

“Two static examples of what the algorithm output looks like. On the left, an 80×80 grid with the source in the corner – the heatmap shows how the distance field is sculpted by random terrain. On the right, an Erdős–Rényi graph drawn with a circular layout, vertex colour encoding distance from the source. The grey vertices in both are unreachable. This is the verification view – before we benchmarked anything, we used these plots to confirm the algorithms agree pixel-for-pixel.”

12. Scaling with V

“Two plots, same data, both axes logarithmic. Left: wall time vs V . Dijkstra is on top, the BFS variants run roughly parallel below. Constant vertical gap on a log scale equals a constant multiplicative speed-up. Right: the speed-up itself. The deque-based 0–1 BFS peaks at around eleven times faster than Dijkstra at $V = 10^8$, then dips back as we move into the memory-bound regime. The other BFS variants cluster in the three-to-five-times range.”

13. Results table (grid, 1 thread)

“Walking the table top to bottom. At a million vertices, 4.6 times faster. Ten million, 5.6. One hundred million with $k = 1$, the deque-based 0–1 BFS hits eleven point one times faster – that’s the peak. The fourth row is the interesting one: at half a billion vertices both algorithms become memory-bandwidth bound and the gap collapses to 0.96 – essentially a tie. Bottom row, our headline: on nine hundred million vertices, 0–1 BFS in 121 seconds, Dijkstra in 620. Every single distance array in the sweep was cross-validated against Dijkstra – zero mismatches.”

14. Where it breaks: k and memory

“Honest about the limits. First, the crossover in k . At a million vertices: $k = 1$ gives 4.6 times, $k = 4$ gives 4.1, $k = 16$ gives 3.3. Once k grows on the order of $\log V$, the asymptotic edge disappears. Second, the memory-bandwidth ceiling at $V \gtrsim 5 \cdot 10^8$: both algorithms become memory-bound, peak pushes hit six hundred million, and a cache-aware Dijkstra catches up. Third, deque versus queues: at $V = 10^8$ with $k = 1$, the deque variant takes 5.4 seconds, the two-queue variant takes 28. Same complexity – the gap is pure allocator and cache overhead from maintaining two separate `std::queues` instead of one `std::deque`.”

15. Repository layout

“Quick tour. `include/zkbfs/` – header-only algorithms and graph types: `common`, `graph_csr`, `grid_graph`, `dijkstra`, `bfs01`, `bfs0k`. `src/` – three CLI executables: `main_bench` for benchmarks, `main_dump` for visualizations, `main_trace` for the instrumented step-by-step. `viz/` – Python for the sweep driver, plots, GIF animations. Build with plain `make`, or `cmake -S . -B build && cmake -build build`.”

16. Next steps (Week 2)

“Four items in the plan. First: an indexed 4-ary heap Dijkstra with real `decrease_key` as a fairer baseline. Second: a chunked ring-buffer FIFO for `bfs_0k` to close the cache gap against the deque-based 0–1 BFS. Third: load a real road network – DIMACS USA – with weights quantised to $k = 8$. Fourth, stretch goal: a Δ -stepping variant for parallel context. Already built in: the 32-bit distance variant, valid whenever $k \cdot V < 2^{32}$. That’s exactly what lets $V \approx 10^9$ fit in 32 GB of RAM. Thank you – questions?”

Likely questions:

- *Why is 0–1 BFS faster than 0– k BFS at $k = 1$?* – One `std::deque` versus two `std::queues`: allocator and cache overhead.
- *How did you verify correctness on a billion-vertex run?* – Distance-sum checksums match Dijkstra exactly; the sum pins down the array without per-vertex comparison.
- *What about parallelism?* – Single thread throughout Stage 1; Δ -stepping is planned for Week 2.